



Sanjivani Rural Education Society's
Sanjivani College of Engineering, Kopargaon-423 603
(An Autonomous Institute, Affiliated to Savitribai Phule Pune University, Pune)
NAAC 'A' Grade Accredited, ISO 9001:2015 Certified

Department of Computer Engineering

(NBA Accredited)

Subject- Object Oriented Programming (CO212)
Unit 4 – Templates and Exception Handling
Topic – 4.1 Function Templates, Overloading Function Templates

Prof. V.N.Nirgude
Assistant Professor

E-mail :

nirgudevikascomp@sanjivani.org.in

Contact No: 9975240215



Introduction

- Template enable us to define generic classes and functions and thus provides support for generic programming.
- Generic programming is an approach where generic types are used as parameters in algorithms so that they work for a variety of data types.



Introduction

- A template can be used to create a family of classes or functions.
- For eg: a class template for an array class would enable us to create arrays of various data types such as: int, float etc.
- Templates are created as parameterized class templates or function templates.
- Template is a simple process to create a generic class with an **anonymous type**.



Function Template

- Function templates are used to create a family of functions with different argument types.
- Syntax:

```
template <class T>
```

```
returntype functionname (arguments of type T)
```

```
{
```

```
.....
```

```
.....
```

```
}
```



Function Template

```
template <class T>
void swap (T x, T y)
{
    T temp = x;
    x = y;
    y = temp;
}
```

```
int main()
{
    fun(100, 200, 11.22, 33.44);
    return 0;
}
```

```
void fun(int m, int n, float a, float b)
{
    swap(m, n);
    swap(a, b);
}
```



Function Template with Multiple Parameters

- We can have more than one generic data type in the function template.

```
template < class T1, class T2>
```

```
returntype functionname(arguments of type T1, T2...)
```

```
{
```

```
..... (Body of function)
```

```
.....
```

```
}
```



Function Template with Multiple Parameters

```
template <class T1, class T2>
void display(T1 x, T2 y)
{
    cout<<x <<" " << y << "\n";
}
int main()
{
    display(1999, "XYZ");
    display (12.34, 1234);
    return 0;
}
```

Output:
1999 XYZ
12.34 1234



Overloading of Function Template

- A template function may be overloaded either by template functions or ordinary functions of its name.
- The overloading is accomplished as follows:
 - Call an ordinary function that has an exact match.
 - Call a template function that could be created with an exact match.
 - Try normal overloading to ordinary function and call the one that matches.



Overloading of Template Functions

```
template < class T>
void display(T x)
{
    cout<<"Template Display : " << x << "\n";
}
void display(int x)
{
    cout << "Explicit Display: " << x << "\n";
}
int main()
{
    display(100);
    display(12.34);
    display("C");
    return 0;
}
```

Output:

Explicit Display:100
Template Display :12.34
Template Display :C



Non-Type Template Arguments

- It is also possible to use non-type arguments.
- In addition to the type argument T, we can also use other arguments such as strings, int, float, built-in types.
- Example:

```
template <class T, int size>
class array
{
    T a[size];
    .....
    .....
};
```



Non-Type Template Arguments

- This template supplies the size of the array as an argument to template.
- The argument must be specified whenever a template class object is created.
- Example:
 - `array <int, 10> a1;     // Array of 10 integers`
 - `array <float, 5> a2;     // Array of 5 floats`
 - `array <char, 20> a3;     // String of size 20`

Thank You..